

Lab 3-1: 開發 tcprtt

練習 1：使用 ring buffer 開發，完成 **tcprtt.c** 和 **tcprtt.bpf.c** 程式

練習 2：編譯並執行 tcprtt。觀察輸出的 ip、port 數值是否正確。在本機可以透過命令列工具簡單製造 tcp 連線

```
# server
```

```
$ python3 -m http.server
```

```
# client
```

```
$ curl 0.0.0.0:8000
```

```
ubuntu@practice-vm:~/Desktop/lab/src/tcprtt$ sudo ./tcprtt
PID      COMM      SRC          DST          LAT(ms)
0        swapper/2  10.0.2.15    :49768 --> 13.107.213.59 :443 26.40
0        swapper/2  10.0.2.15    :49768 --> 13.107.213.59 :443 26.40
30       ksoftirqd/2 10.0.2.15    :60426 --> 13.107.246.59 :443 28.10
30       ksoftirqd/2 10.0.2.15    :60426 --> 13.107.246.59 :443 28.10
6508     curl       127.0.0.1    :8000 --> 127.0.0.1      :35264 0.01
6508     curl       127.0.0.1    :35264 --> 127.0.0.1      :8000 0.02
6506     python3    127.0.0.1    :35264 --> 127.0.0.1      :8000 0.04
6506     python3    127.0.0.1    :8000 --> 127.0.0.1      :35264 0.01
6506     python3    127.0.0.1    :35264 --> 127.0.0.1      :8000 0.04
6506     python3    127.0.0.1    :8000 --> 127.0.0.1      :35264 0.04
6506     python3    127.0.0.1    :35264 --> 127.0.0.1      :8000 0.04
```

```
ubuntu@practice-vm:~$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [16/Nov/2025 13:48:32] "GET / HTTP/1.1" 200 -
```

```
ubuntu@practice-vm:~$ curl 0.0.0.0:8000
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
```

Lab 3-2: 開發 tcprtt tp

- 練習 1：使用 hash map 開發，完成 **tcprtt_tp.bpf.c** 程式
- 練習 2：編譯並執行後，確認輸出的 ip、port 數值正確。

```
57 | goto destroy;
58 | }

46 | if (!rb) {
    | ^
1 error generated.
make: *** [Makefile:15: tcprrt_tp] Error 1
● ubuntu@practice-vm:~/lab/src/tcprrt$ make tcprrt_tp
  clang -g tcprrt_tp.c -lbpf -lelf -lz -o tcprrt_tp
○ ubuntu@practice-vm:~/lab/src/tcprrt$ sudo ./tcprrt_tp
PID   COMM      SRC          DST          :8000    LAT(ms)
7316   curl       127.0.0.1    :53916 --> 127.0.0.1    :8000    0.04
7316   curl       127.0.0.1    :8000  --> 127.0.0.1    :53916   0.00
0      swapper/2  10.0.2.15    :44860 --> 13.107.213.59 :443     54.28
```

fentry 與 tracepoint 的差異

tracepoint 主要用於監控特定事件發生的時機，而不會直接進入函式執行。fentry 與 tracepoint 的差異對 tcprrt 程式表現的影響在 tcprrt 程式中，使用 tracepoint 通常表現更好，因為 tracepoint 是基於內核事件（如 TCP 連線狀態變更）觸發的，能在適當的時機捕捉資料，減少不必要的操作。相比之下，fentry 掛勾的是內核函式的進入，這會使得每次目標函式調用時都觸發探針，可能帶來較高的性能開銷，特別是當函式被頻繁調用時。因此，tracepoint 更加高效，對 tcprrt 的性能影響較小。